



\$ macOS detecting race-conditions

Olivia Gallucci @ Datadog
Black Hat

prev. Apple, SECUINFRA GmbH, U.S. Govt.
Aug 2026

\$ whatis vocabulary

Processes are runnable entities.

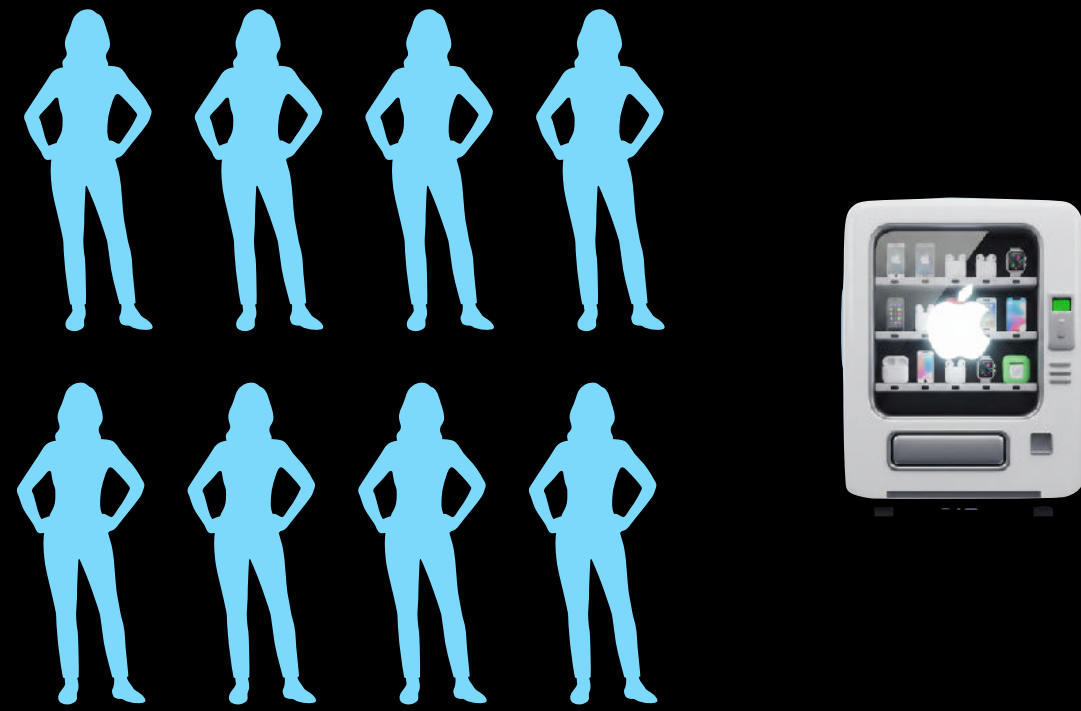
Processes are *not* runnable entities.

The process itself is technically **only a container** for one or more threads, and **provides** the virtual memory image, the descriptors and ports **shared by all the threads**.

Thus, when one refers to a process as “**executing,**” the correct terminology is “**at least one of the threads of the process pid is executing.**”



Concurrent: 2 queues, 1 vending machine



Concurrent: 2 queues, 1 vending machine

Interleavings

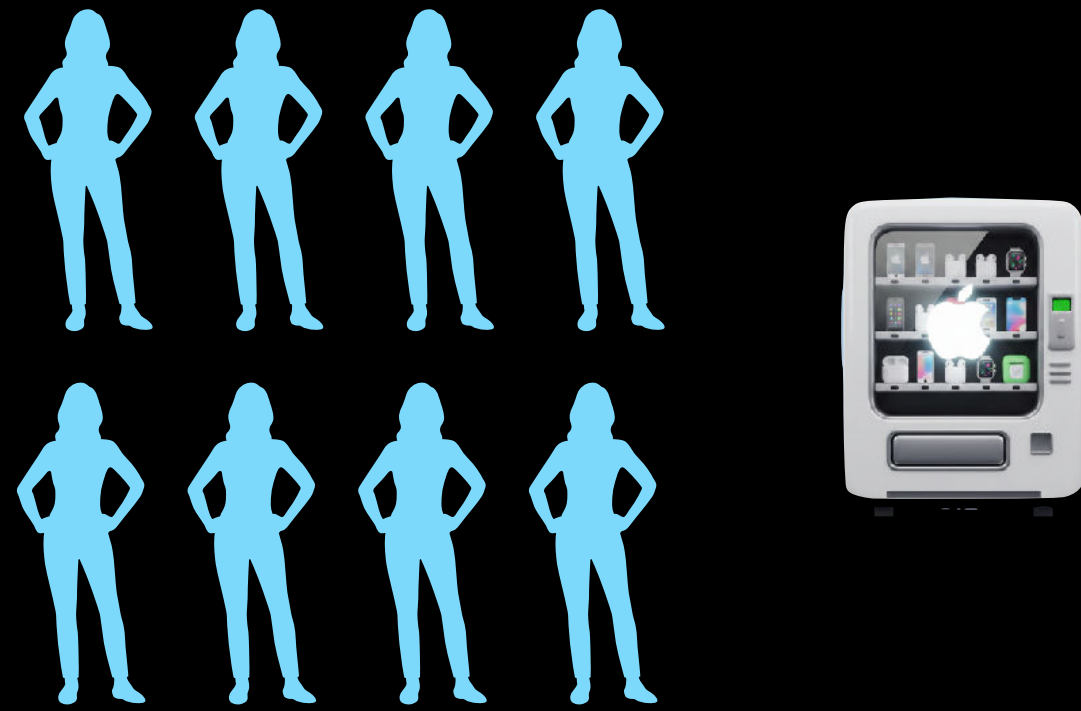
Time ↓

```
T1:  A: read x
      B: x = x + 1
      C: write x

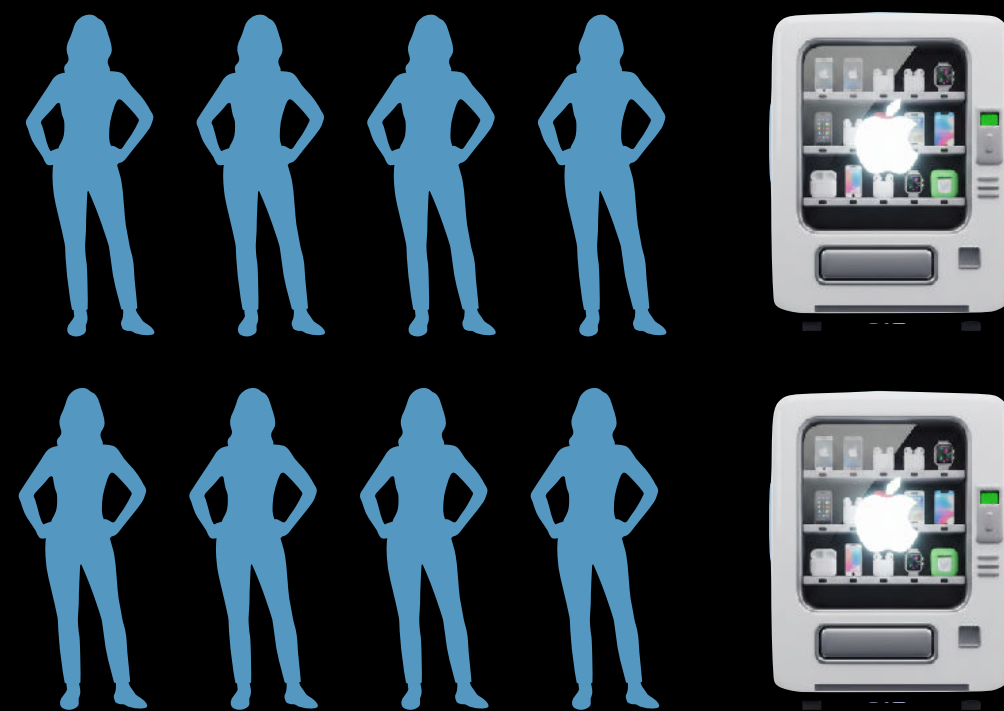
T2:      D: read x
          E: x = x + 1
          F: write x
```

One Possible Interleaving

```
T1:  A      B      C
T2:      D      E      F
```

Concurrent: 2 queues, 1 vending machine



Parallel: 2 queues, 2 vending machines

Interleavings

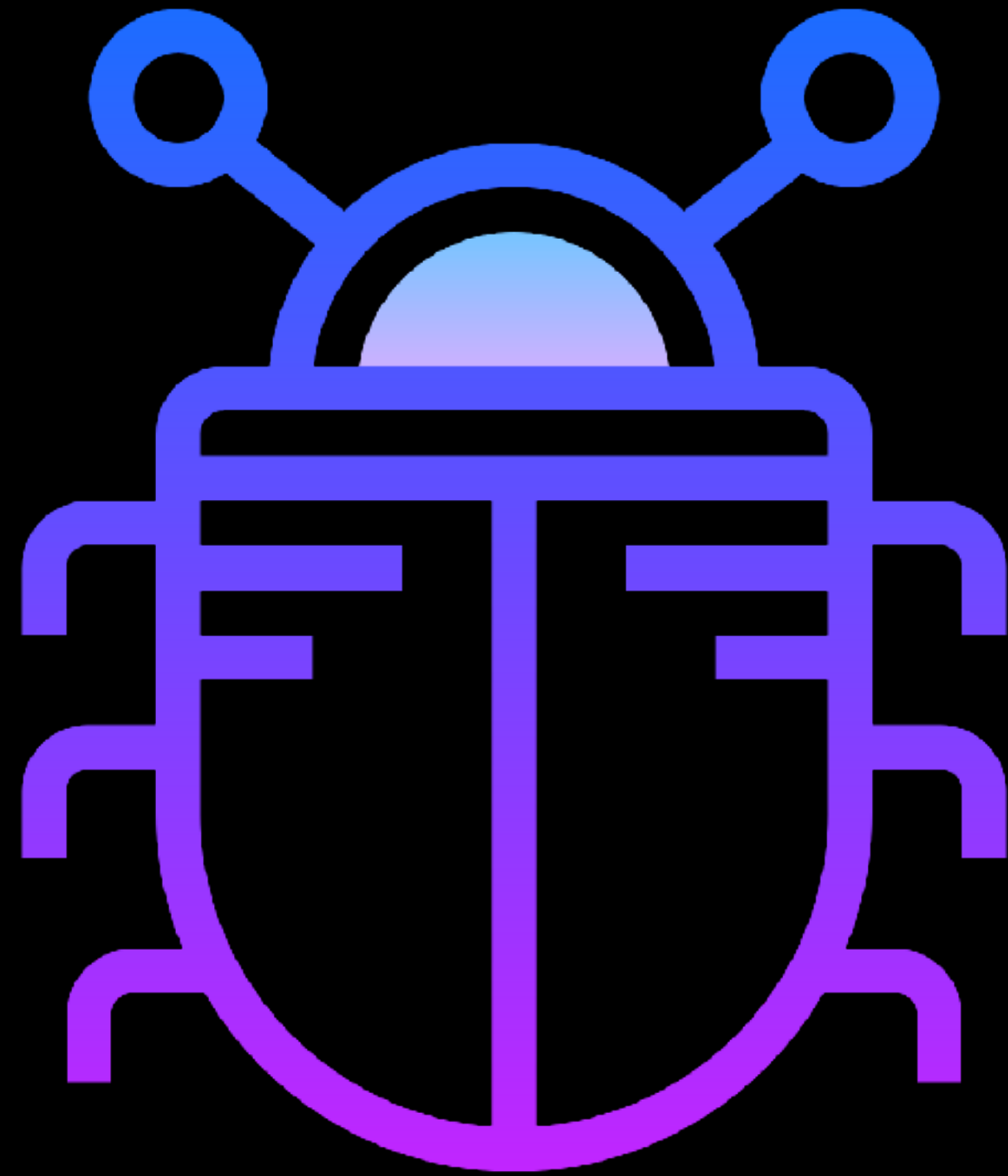
Time ↓

T1: A: read x
 B: $x = x + 1$
 C: write x

T2: D: read x
 E: $x = x + 1$
 F: write x

One Possible Interleaving

T1:	A		B		C	
T2:		D		E		F



Concurrency



Parallelism

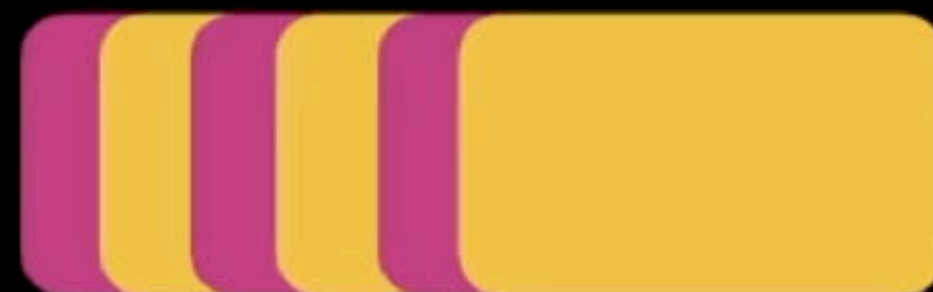
```
$ ssh gcd@apple.com
```


Grand Central Dispatch

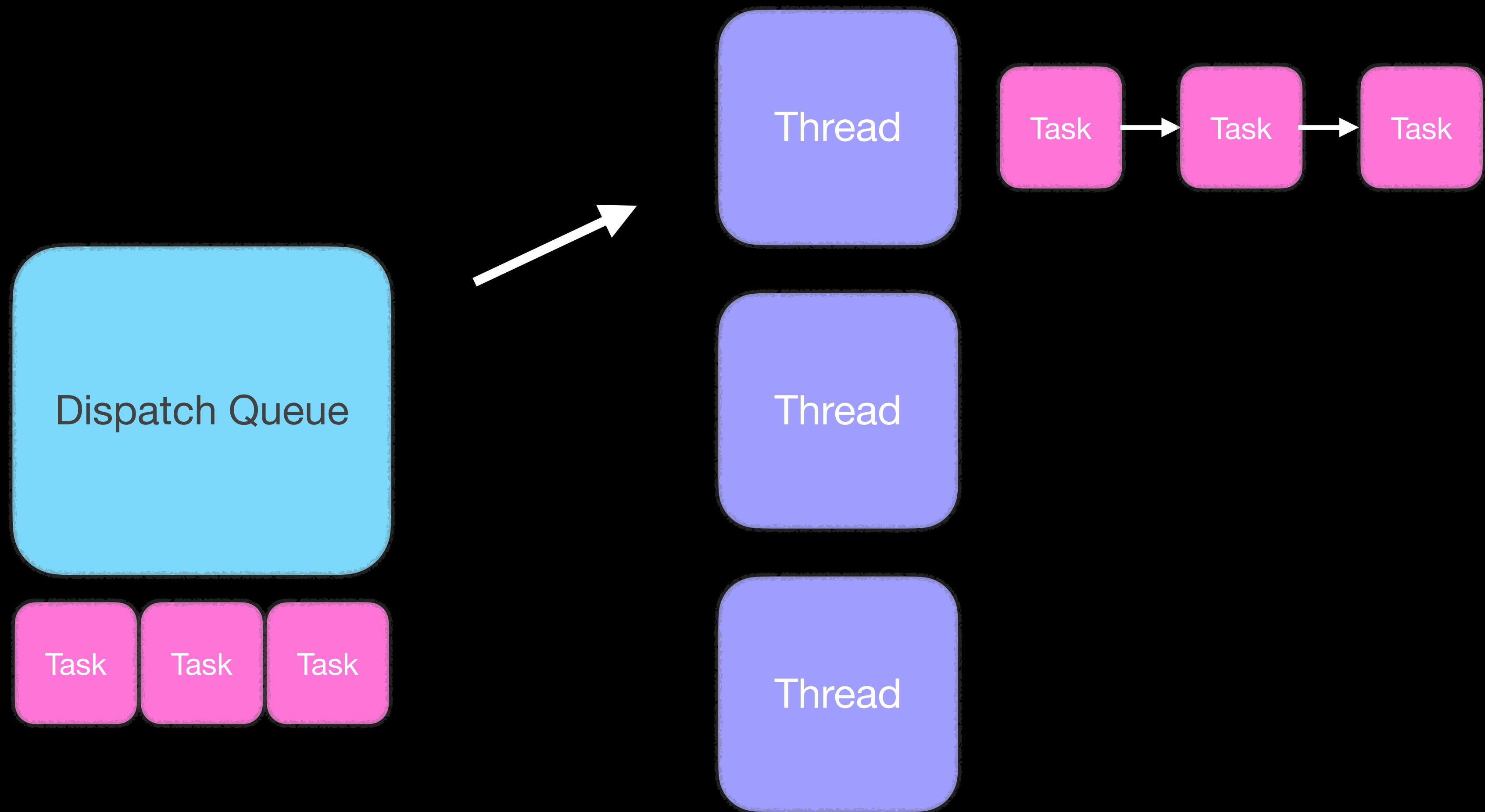
Task Queue



Thread Pool



Completed Tasks



AppleDeveloper

Get Started

Platforms

Technologies

Community

Documentation

Downloads

Support

Documentation

Language: Swift

< All Technologies

Dispatch

Queues and Tasks

> [C] DispatchQueue

> [C] DispatchWorkItem

> [C] DispatchGroup

> [≡] Dispatch Queue

> [≡] Dispatch Work Item

> [≡] Dispatch Group

> [≡] Workloop

Thread Scheduling

> [S] DispatchQoS

System Event Monitoring

> [C] DispatchSource

> [≡] Dispatch Source

> [C] DispatchIO

> [S] DispatchData

> [S] DispatchDataIterator

> [≡] Dispatch I/O

> [≡] Dispatch Data

> [Pr] DispatchSourceProtocol

Filter

/

Framework

Dispatch

Execute code concurrently on multicore hardware by submitting work to dispatch queues managed by the system.

iOS 8.0+ | iPadOS 8.0+ | Mac Catalyst 13.0+ | macOS 10.10+ | tvOS 9.0+ | visionOS 1.0+ | watchOS 2.0+

Overview

Dispatch, also known as Grand Central Dispatch (GCD), contains language features, runtime libraries, and system enhancements that provide systemic, comprehensive improvements to the support for concurrent code execution on multicore hardware in macOS, iOS, watchOS, and tvOS.

The BSD subsystem, Core Foundation, and Cocoa APIs have all been extended to use these enhancements to help both the system and your application to run faster, more efficiently, and with improved responsiveness. Consider how difficult it is for a single application to use multiple cores effectively, let alone to do it on different computers with different numbers of computing cores or in an environment with multiple applications competing for those cores. GCD, operating at the system level, can better accommodate the needs of all running applications, matching them to the available system resources in a balanced fashion.

Dispatch Objects and ARC

When you build your app using the Objective-C compiler, all dispatch objects are Objective-C objects. As such, when automatic reference counting (ARC) is enabled, dispatch objects are retained and released automatically, just like any other Objective-C object. When ARC is not enabled, use the [dispatch_retain](#) and [dispatch_release](#) functions (or Objective-C semantics) to retain and release your dispatch objects. You cannot use the Core Foundation retain and release functions.

Vlog

Olivia Gallucci - Draft - Approved-Use Only

Quality of Service (QoS)



Unspecified

Default

User
Interactive

User
Initiated

Utility

Background

Quality of Service (QoS)

- **User Interactive:** Animations or tasks that update the UI immediately.
- **User Initiated:** Tasks required for a seamless user experience, such as loading data for a scrolling table view.
- **Utility:** Long-running tasks where user is aware of progress (e.g., downloads).
- **Background:** Tasks user is unaware of, such as backups or server restoration.
- **Default & Unspecified:** Default falls between User Initiated and Utility. Unspecified indicates missing QoS information and has the lowest priority.

Unspecified

Default

User
Interactive

User
Initiated

Utility

Background

Public APIs

- Swift/ObjC (Foundation)
 - `QualityOfService` (`userInteractive`, `userInitiated`, `utility`, `background`, plus `default`).
- Swift (Dispatch)
 - `DispatchQoS` / `DispatchQoS.QoSClass` with the same classes
 - System prioritizes higher QoS work when scheduling
- C (libdispatch)
 - queue attributes and global queues take `QOS_CLASS_*` constants (e.g., `QOS_CLASS_USER_INTERACTIVE`, `..._USER_INITIATED`, `..._UTILITY`, `..._BACKGROUND`)

Quality of Service (QoS)

- **User Interactive:** Animations or tasks that update the UI immediately.
- **User Initiated:** Tasks required for a seamless user experience, such as loading data for a scrolling table view.
- **Utility:** Long-running tasks where user is aware of progress (e.g., downloads).
- **Background:** Tasks user is unaware of, such as backups or server restoration.
- **Default & Unspecified:** Default falls between User Initiated and Utility. Unspecified indicates missing QoS information and has the lowest priority.

Unspecified

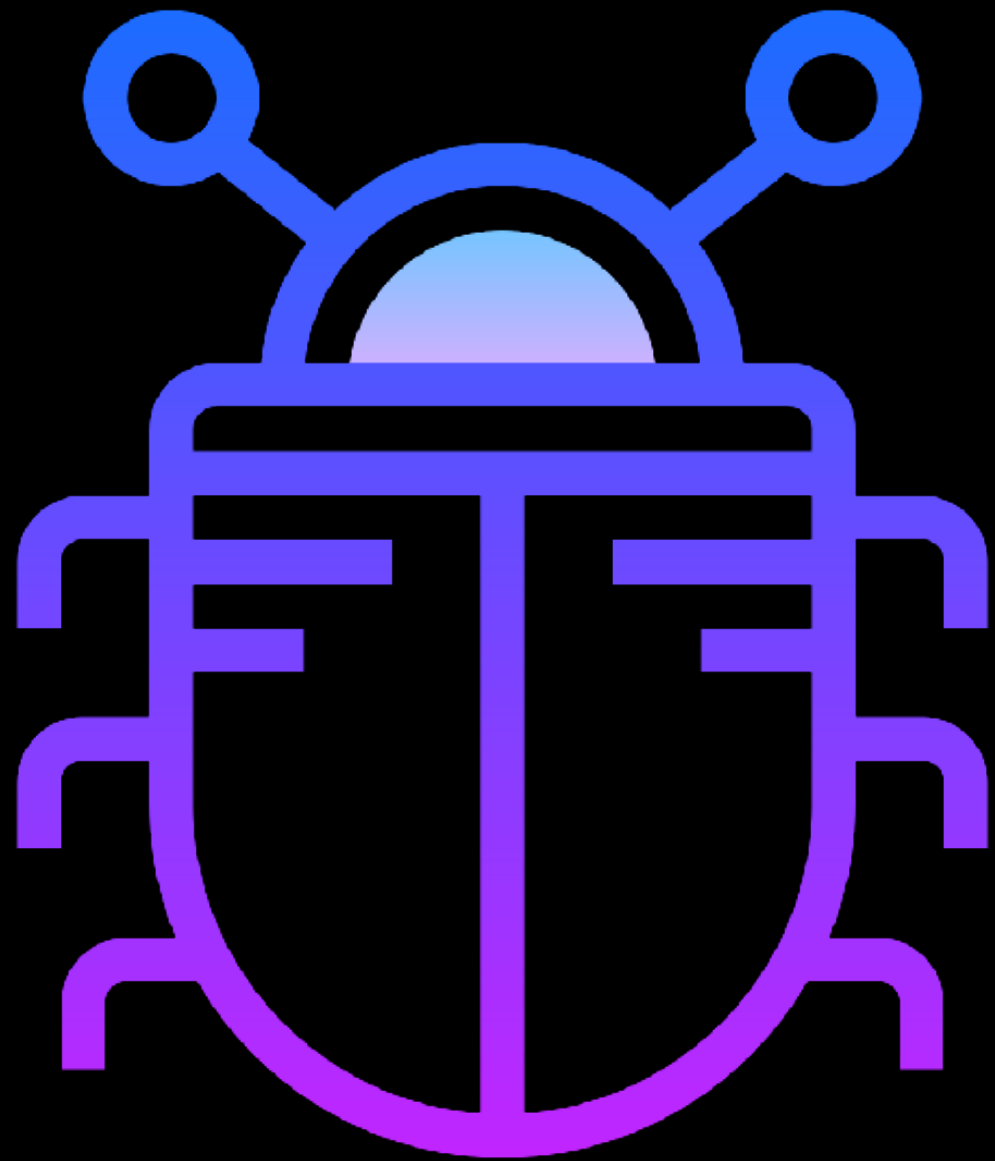
Default

User
Interactive

User
Initiated

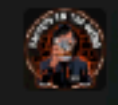
Utility

Background



Podcast Episode

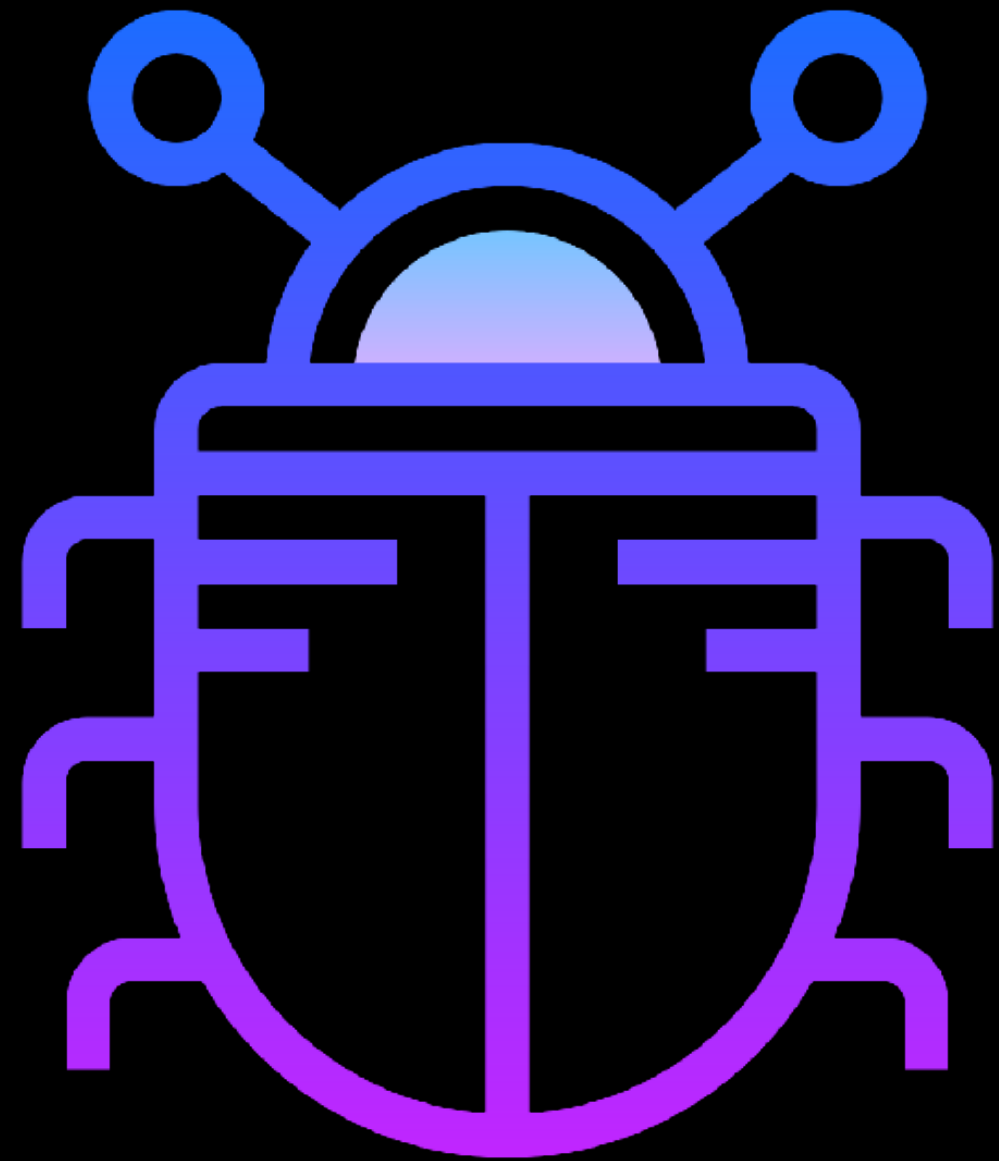
How Hackers Exploit Milliseconds - TOCTOU Attacks

 Hackers On The Rocks

Video • Jul 7, 2025 • 32 min 34 sec

gotta do the self promo

How to manipulate the
execution flow of TOCTOU
attacks ([ret]2read)



```
$ cd /vulnerabilities
```

Priority Inversion and QoS Mismatches

3-Thread Swimlane: L holds lock, H blocks, M runs



high priority



low priority

Priority Inversion and QoS Mismatches

3-Thread Swimlane: L holds lock, H blocks, M runs



high priority



medium priority



low priority

Priority Inversion and QoS Mismatches

3-Thread Swimlane: L holds lock, H blocks, M runs



low priority



high priority

Priority Inversion and QoS Mismatches

3-Thread Swimlane: L holds lock, H blocks, M runs

Assume priorities: $H > M > L$. Shared resource protected by mutex R.



- L acquires R.
- H becomes runnable, needs R, blocks on L.
- While H is blocked, M runs (preempts L), so L can't finish and release R.
- Result: H is indirectly delayed by M (lower priority than H), which is the inversion.

Does Apple implement priority inheritance anywhere?

Next few slides are a simplification, but gets the point across

mutex vs unfair

unfair vs spinlock

spinlock vs semaphores

pthread_mutex_t (blocks/sleeps, scheduler can help)

```
static pthread_mutex_t gMutex = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&gMutex);    // contended: thread can sleep
// critical section
pthread_mutex_unlock(&gMutex);
```

os_unfair_lock (Darwin replacement for spin)

```
static os_unfair_lock gLock = OS_UNFAIR_LOCK_INIT;

os_unfair_lock_lock(&gLock);    // contended: waiter parks
// critical section
os_unfair_lock_unlock(&gLock);
```


mutex vs unfair

unfair vs spinlock

spinlock vs semaphores

pthread_mutex_t (blocks/sleeps, scheduler can help)

```
static pthread_mutex_t gMutex = PTHREAD_MUTEX_INITIALIZER;  
  
pthread_mutex_lock(&gMutex);    // contended: thread can sleep  
// critical section  
pthread_mutex_unlock(&gMutex);
```

os_unfair_lock (Darwin replacement for spin)

```
static os_unfair_lock gLock = OS_UNFAIR_LOCK_INIT;  
  
os_unfair_lock_lock(&gLock);    // contended: waiter parks  
// critical section  
os_unfair_lock_unlock(&gLock);
```


os_unfair_lock (Darwin replacement for spin)

```
static os_unfair_lock gLock = OS_UNFAIR_LOCK_INIT;

os_unfair_lock_lock(&gLock);    // contended: waiter parks
// critical section
os_unfair_lock_unlock(&gLock);
```

Plain spinlock (bad under mixed QoS)

```
static atomic_flag gSpin = ATOMIC_FLAG_INIT;

while (atomic_flag_test_and_set_explicit(&gSpin, memory_order_acquire)) {
    /* spins: keeps running, burns CPU */
}
// critical section
atomic_flag_clear_explicit(&gSpin, memory_order_release);
```


mutex vs unfair

unfair vs spinlock

spinlock vs semaphores

mutex vs unfair

unfair vs spinlock

spinlock vs semaphores

With QoS inheritance (pthread_mutex / os_unfair_lock)

Lock tracks ownership; kernel can temporarily boost the owner to resolve inversion

```
time →  
High QoS (Q=UI) : lock() — BLOCKED ————— RUN (after unlock)  
Low QoS (Q=BG) : RUN — holds lock —(BOOST to UI)— RUN — unlock()  
CPU scheduling : runs boosted BG owner so it can release sooner
```

```
typedef NSObject<OS_dispatch_semaphore> * dispatch_semaphore_t;
```


With QoS inheritance (pthread_mutex / os_unfair_lock)

Lock tracks ownership; kernel can temporarily boost the owner to resolve inversion

```
time →  
High QoS (Q=UI) : lock() — BLOCKED ————— RUN (after unlock)  
Low QoS (Q=BG) : RUN — holds lock —(BOOST to UI)— RUN — unlock()  
CPU scheduling : runs boosted BG owner so it can release sooner
```

Without QoS inheritance (semaphore / rwlock / custom)

Waiter can't donate QoS/priority to the owner, so the high-QoS thread stalls

```
time →  
High QoS (Q=UI) : wait() — BLOCKED —————  
Low QoS (Q=BG) : READY —————(not boosted)————— RUN — signal()  
CPU scheduling : keeps running other work; BG owner may not run promptly
```

* "With QoS inheritance (pthread_mutex / os_unfair_lock)" is really "with **potential** QoS inheritance." Apple's wording is deliberately hedged ("may use to attempt").

mutex vs unfair

unfair vs spinlock

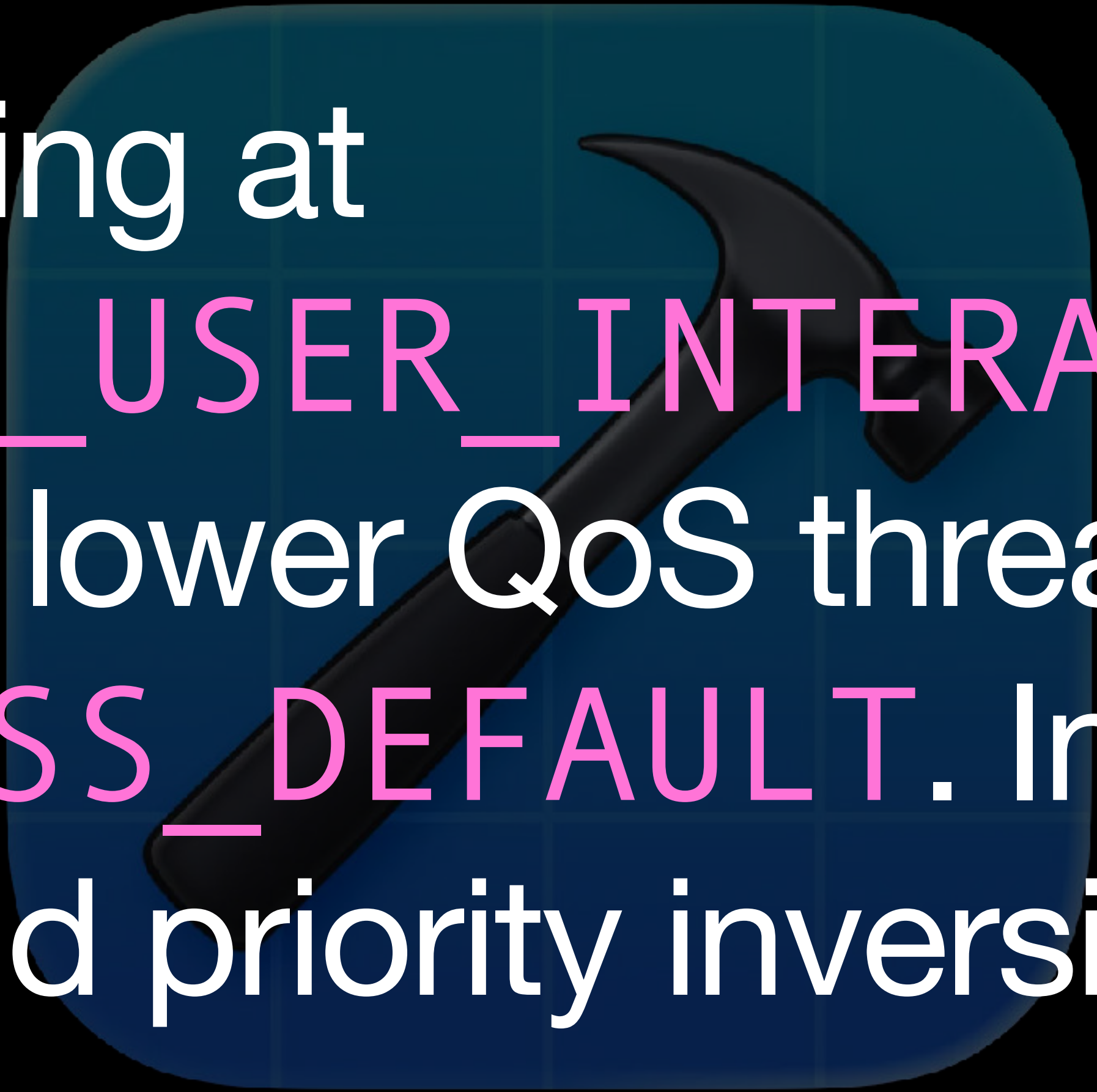
spinlock vs semaphores

./detection





Edit Scheme... > Run > Diagnostics >
Runtime API Checking and check
“Thread Performance Checker”



Thread running at
`QOS_CLASS_USER_INTERACTIVE`
waiting on a lower QoS thread running
at `QOS_CLASS_DEFAULT`. Investigate
ways to avoid priority inversions.

Thread running
QOS_CLASS_
waiting on a
at QOS_CLASS_
ways to avoid
ACTIVE
ad running
investigate
ions.



./dispatch-sync-deadlocks

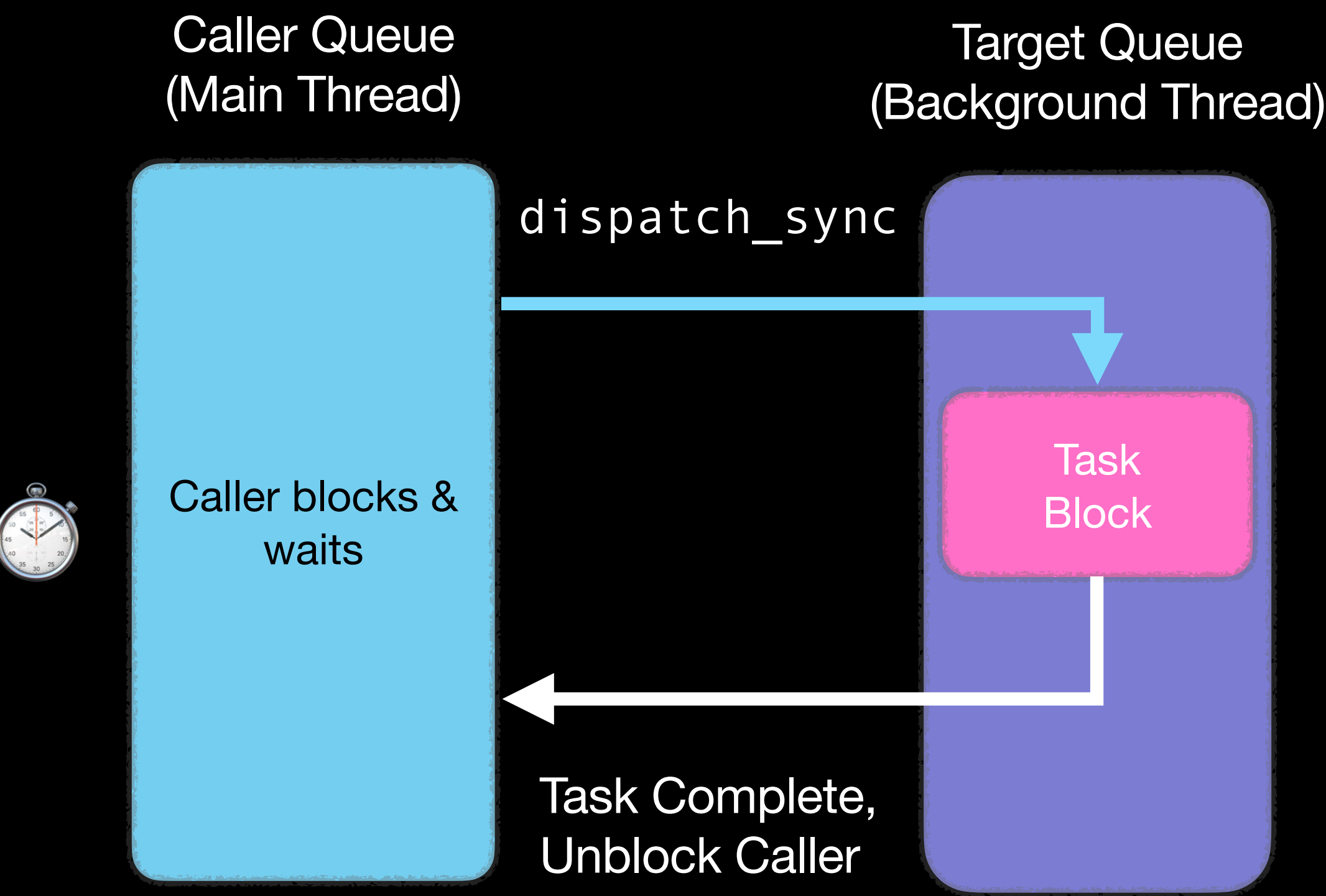
dispatch_sync

circular wait

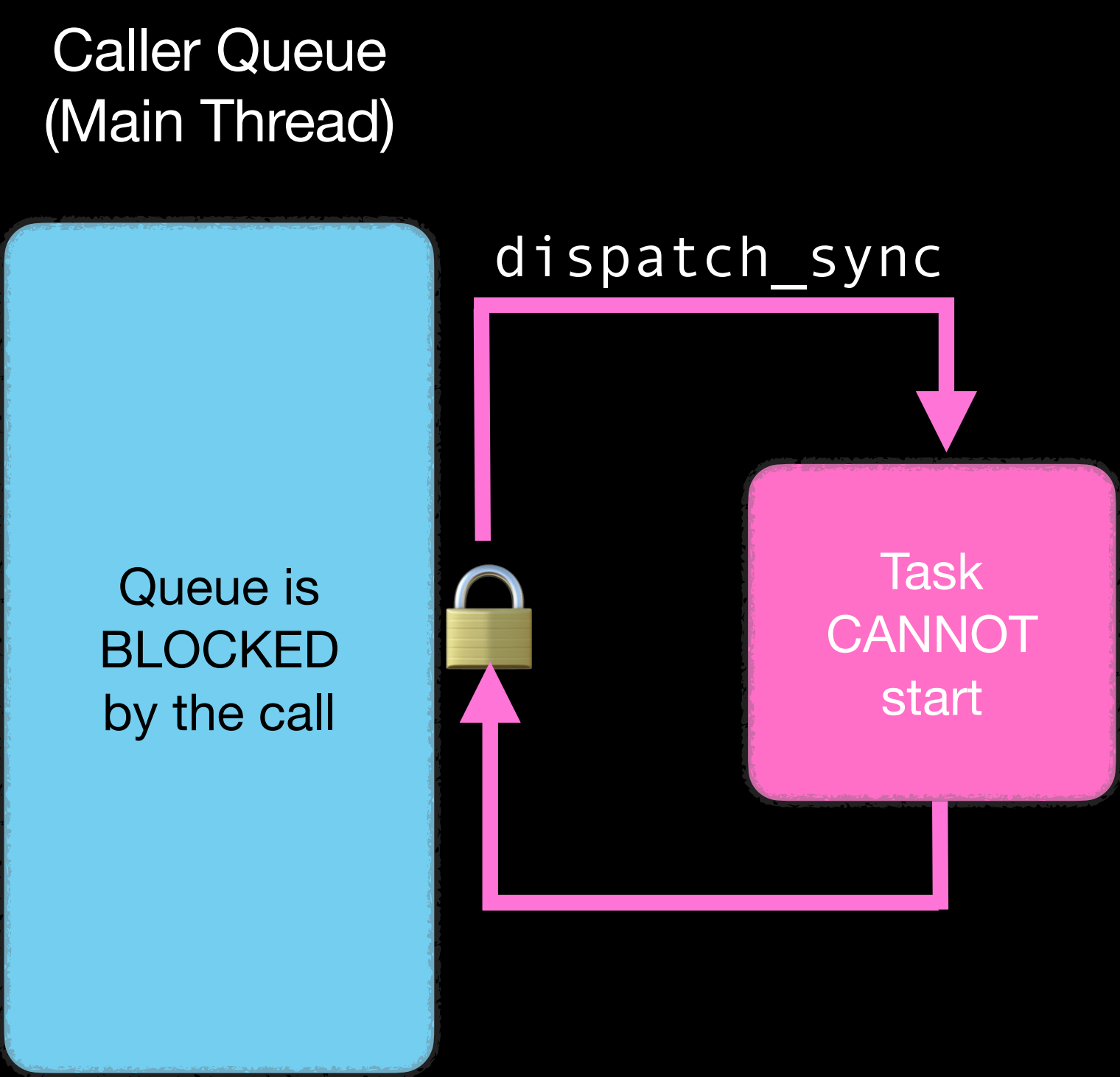
scanning

understanding dispatch_sync & deadlocks

synchronous execution (normal case)



deadlock scenario (problem case)



dispatch_sync
circular wait
scanning

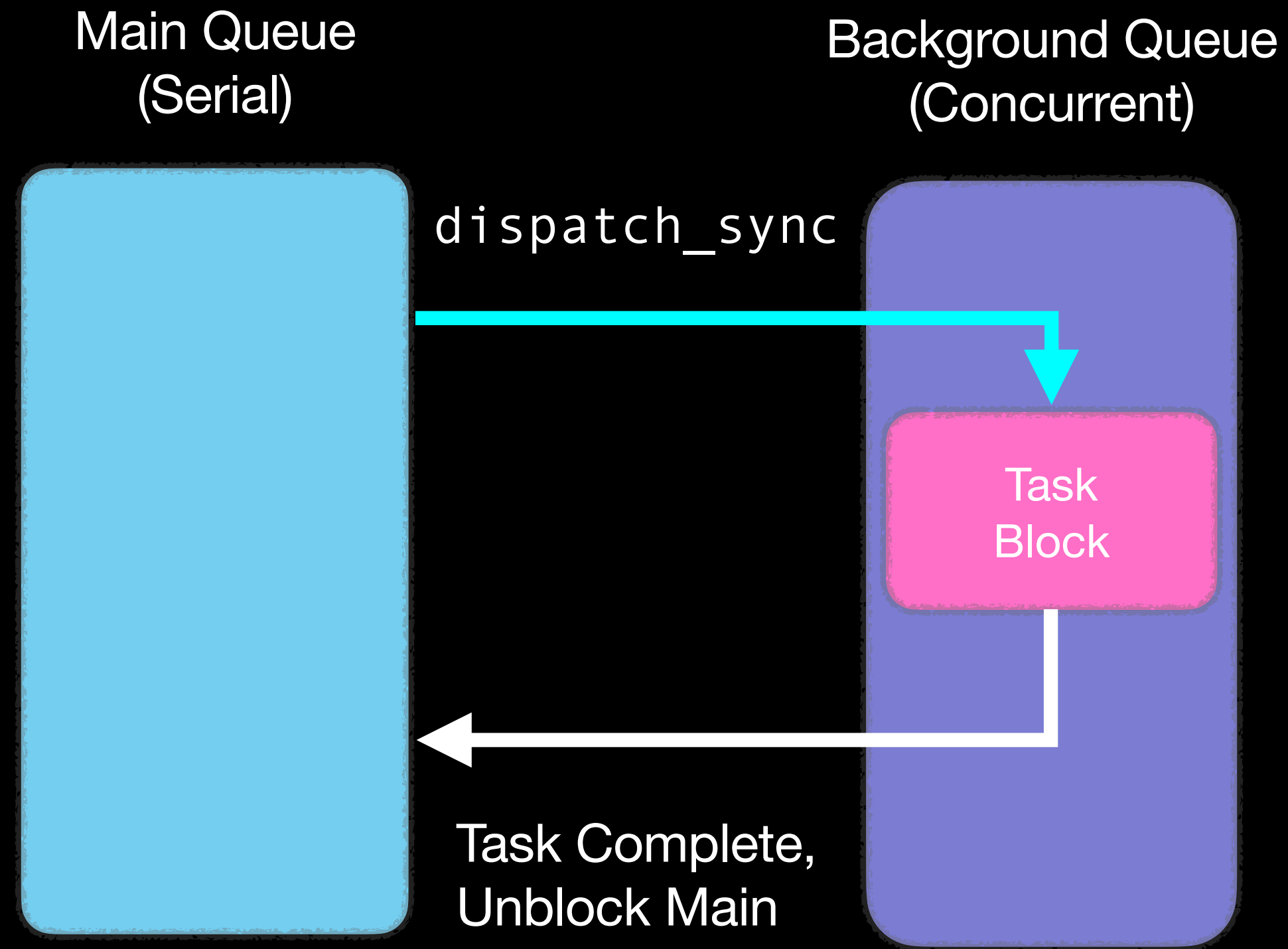


Main Dispatch
Queue

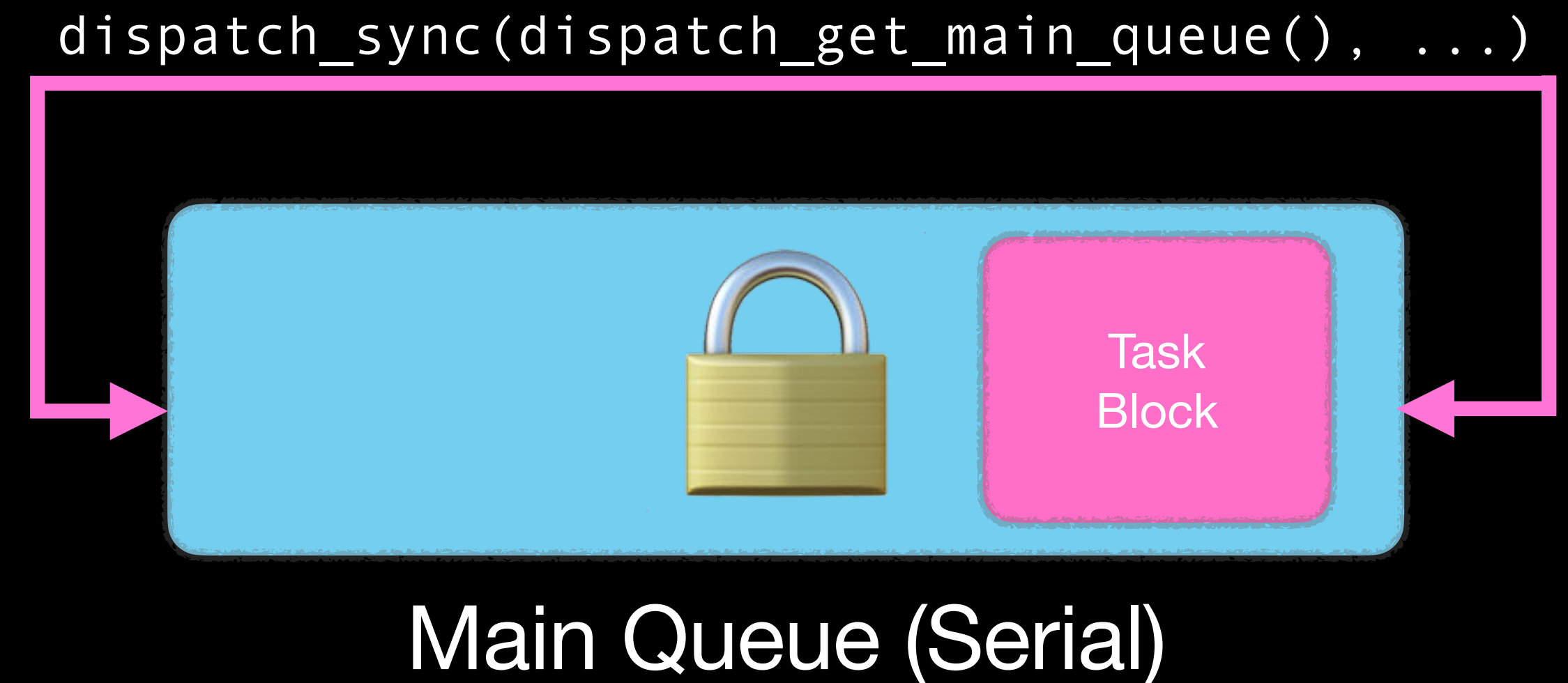
understanding `dispatch_sync` deadlocks

the main queue pitfall

safe: targeting a *different* queue
(e.g., background)

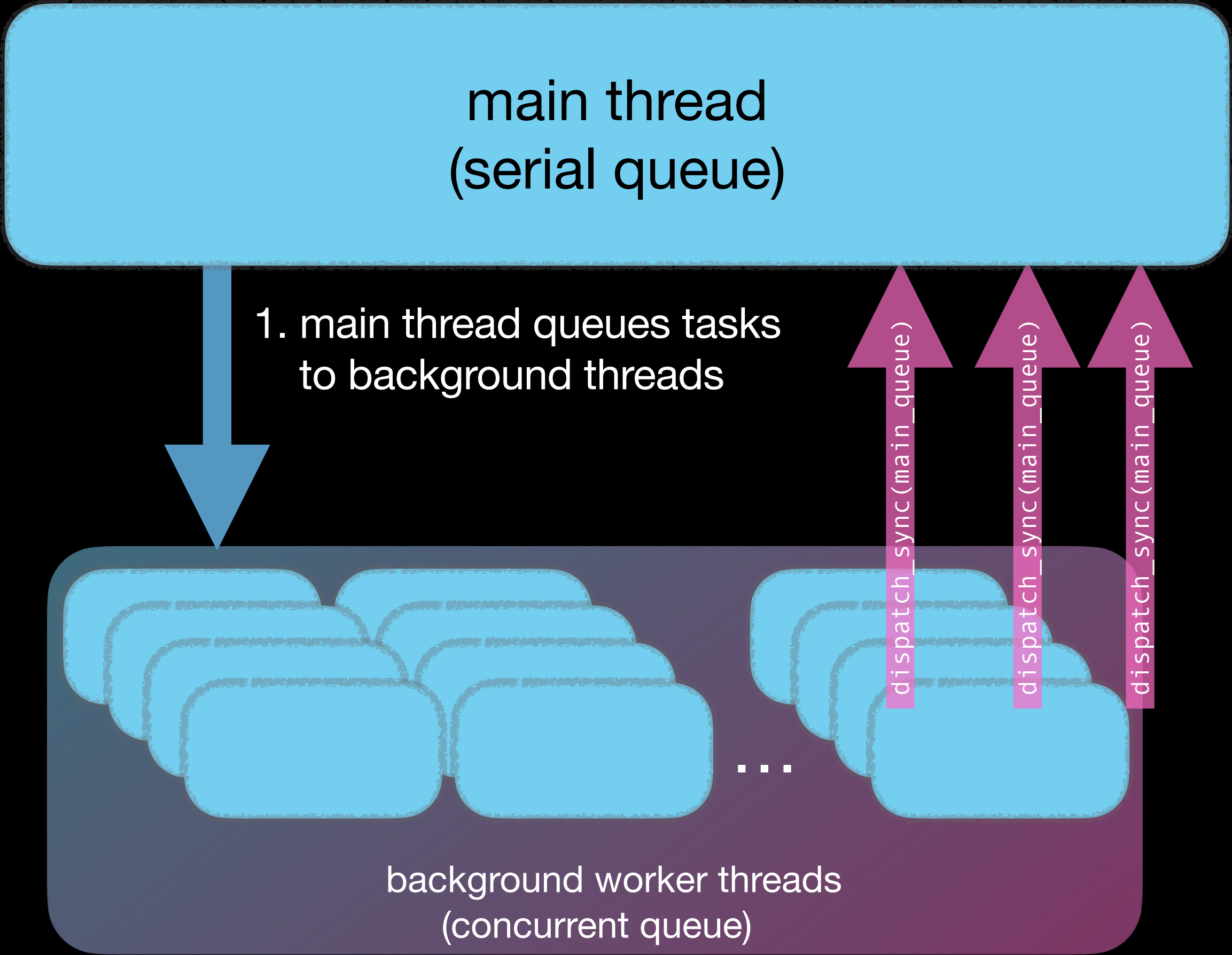


deadlock: targeting the *main* queue from the main thread



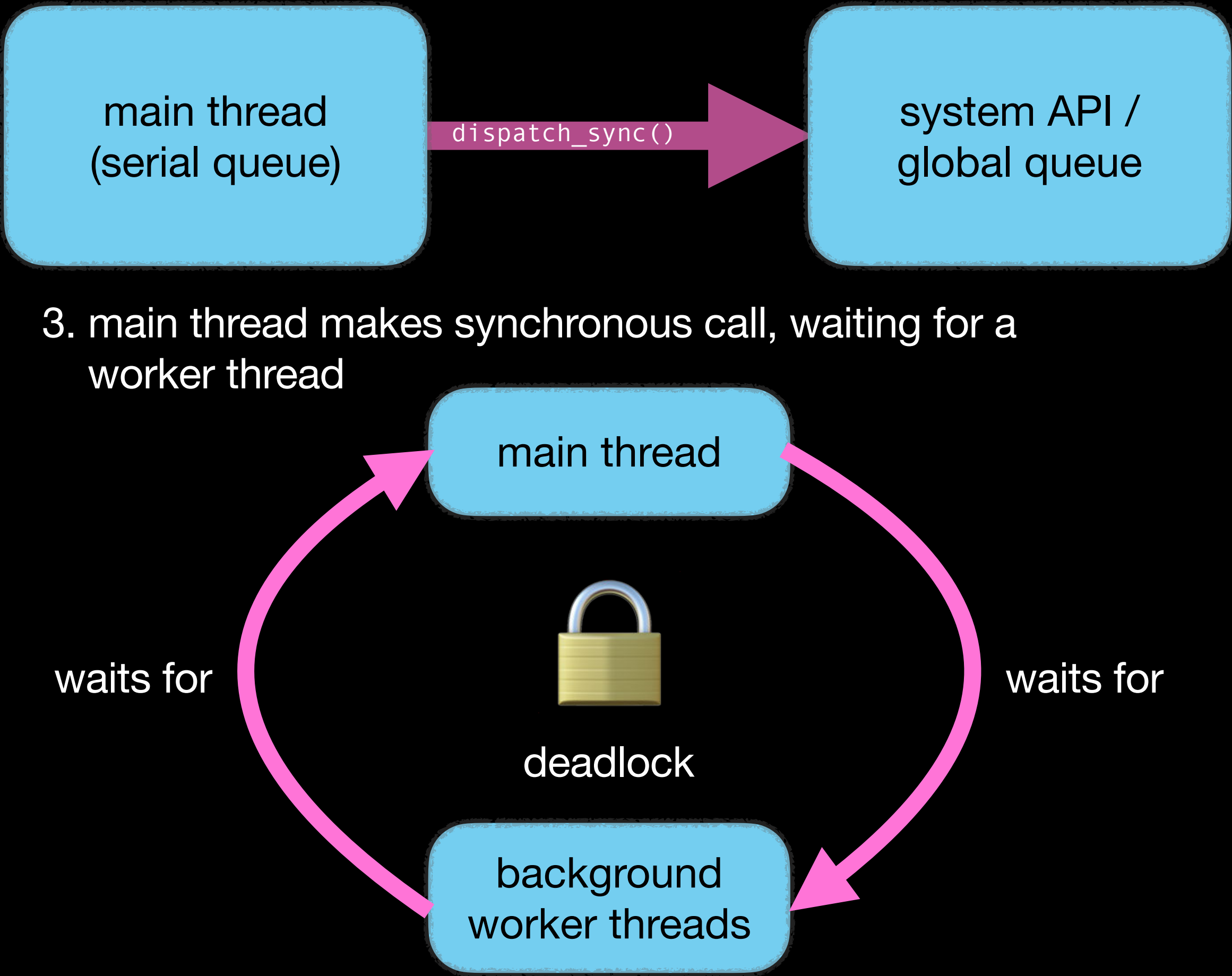
circular wait

setup & trigger



2. background threads block, waiting for main thread

deadlock (circular wait)



4. circular wait: main thread waits for workers, workers wait for main thread. app freezes.

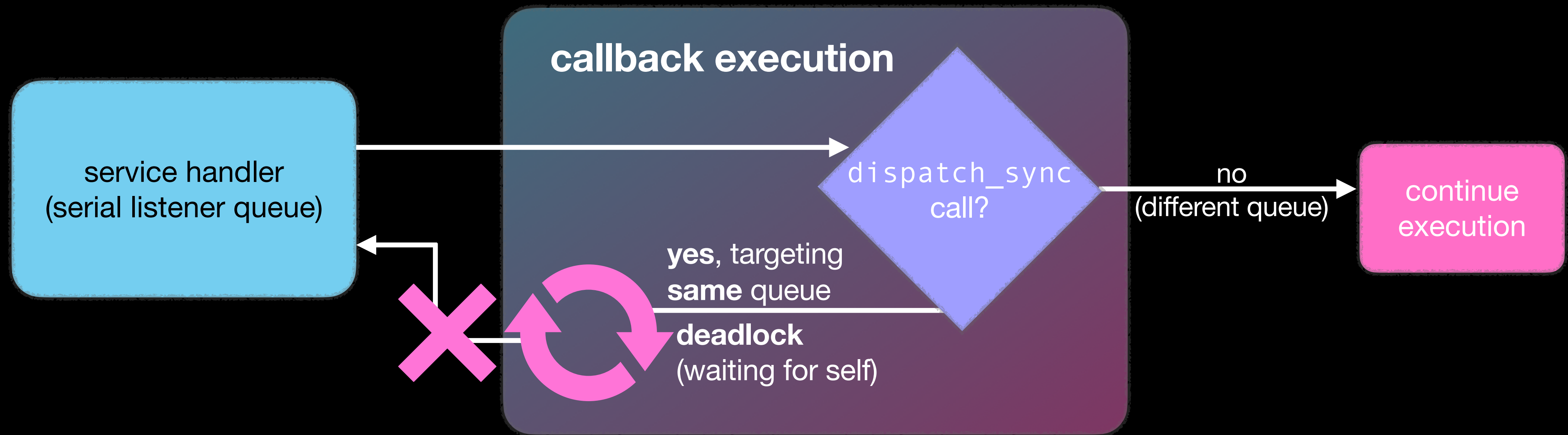
dispatch_sync

circular wait

scanning

deadlock scenario

dispatch_sync on serial queue



guaranteed to deadlock service

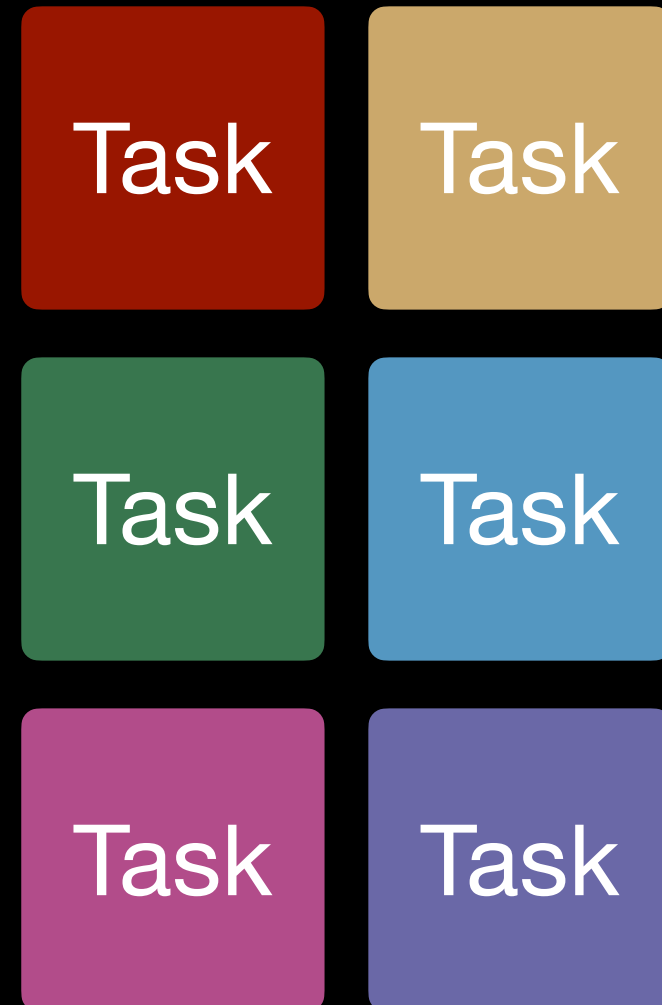
\$ brew uninstall resource-starvation

Resource Starvation & GCD

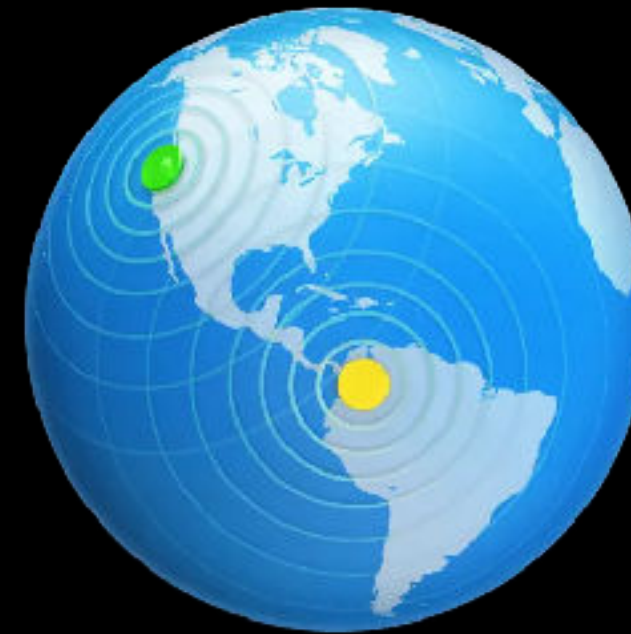
thread pool saturation



DISPATCH_ASYNC
(huge volume / blocking)



concurrent queue tasks

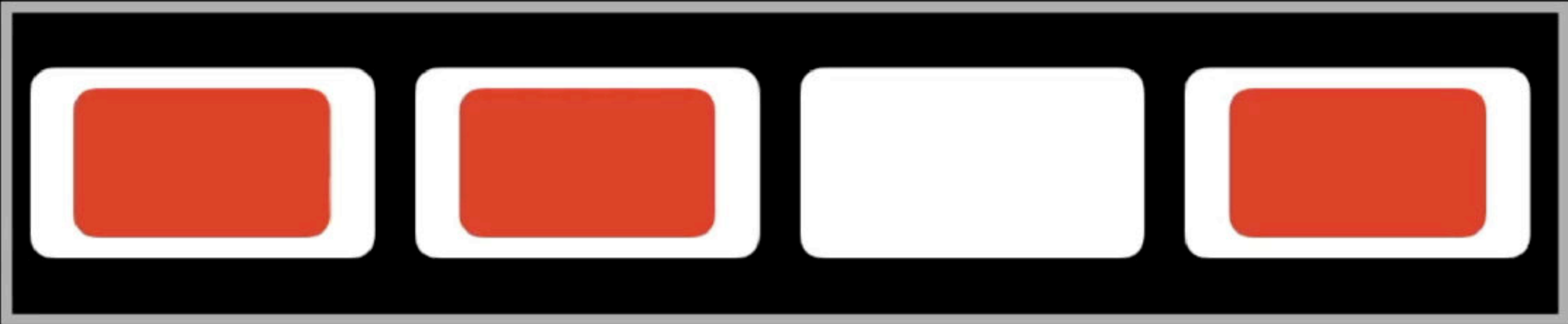


worker thread pool
saturated

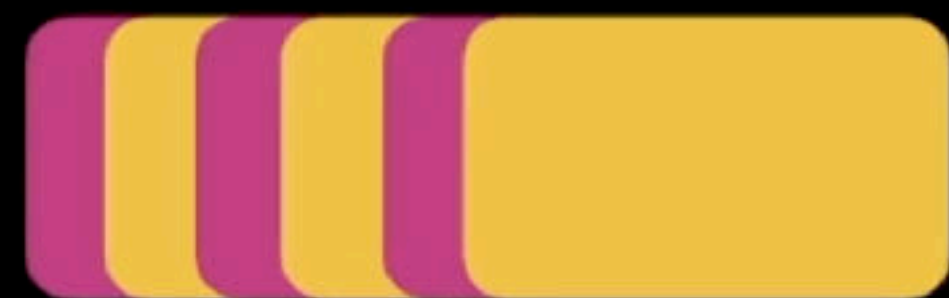
- misusing GCD can lead to resource starvation
- GCD uses a thread pool for concurrent tasks
- saturating the pool staves other tasks of CPU/threads

Grand Central Dispatch

Task Queue

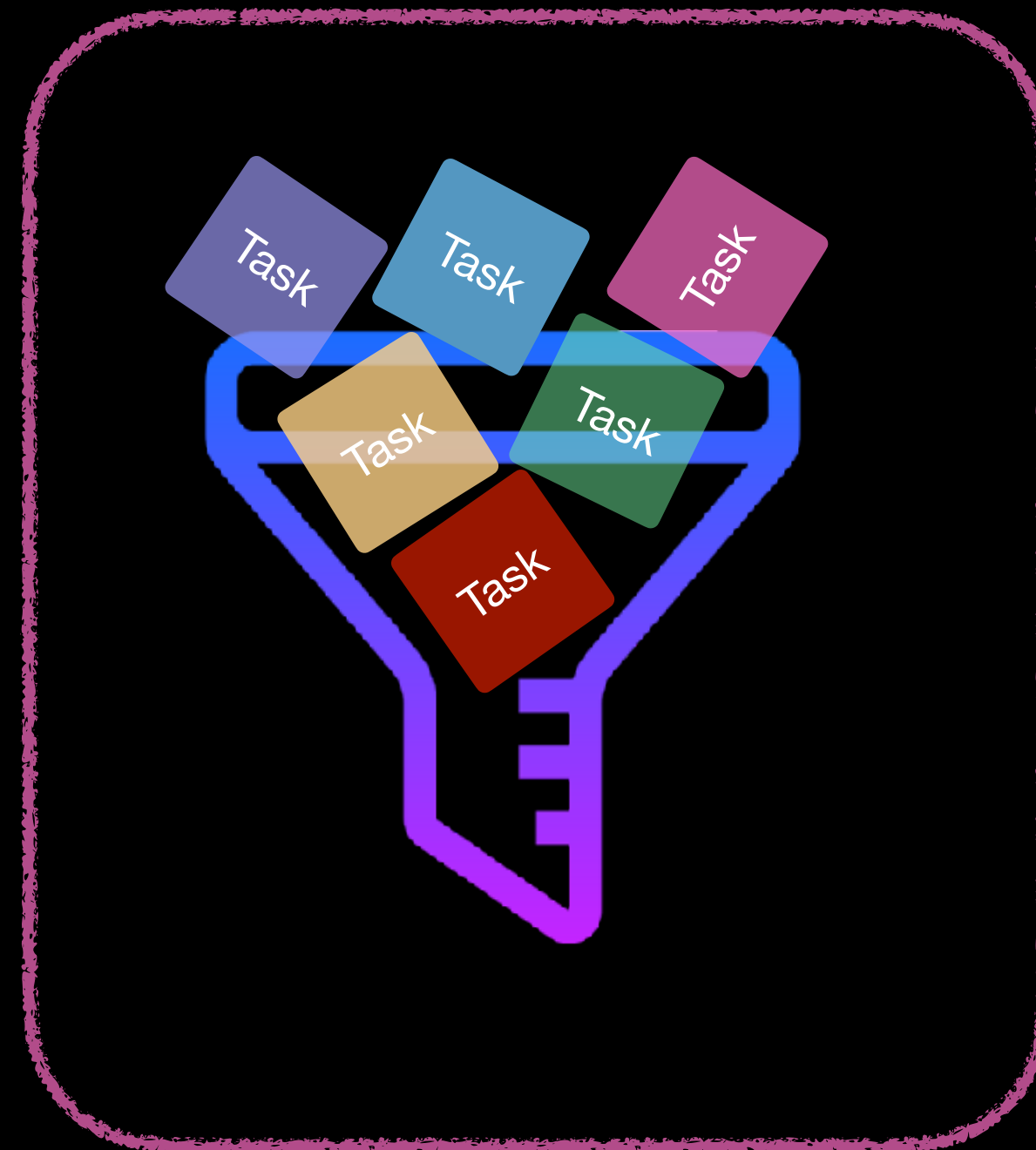


Thread Pool



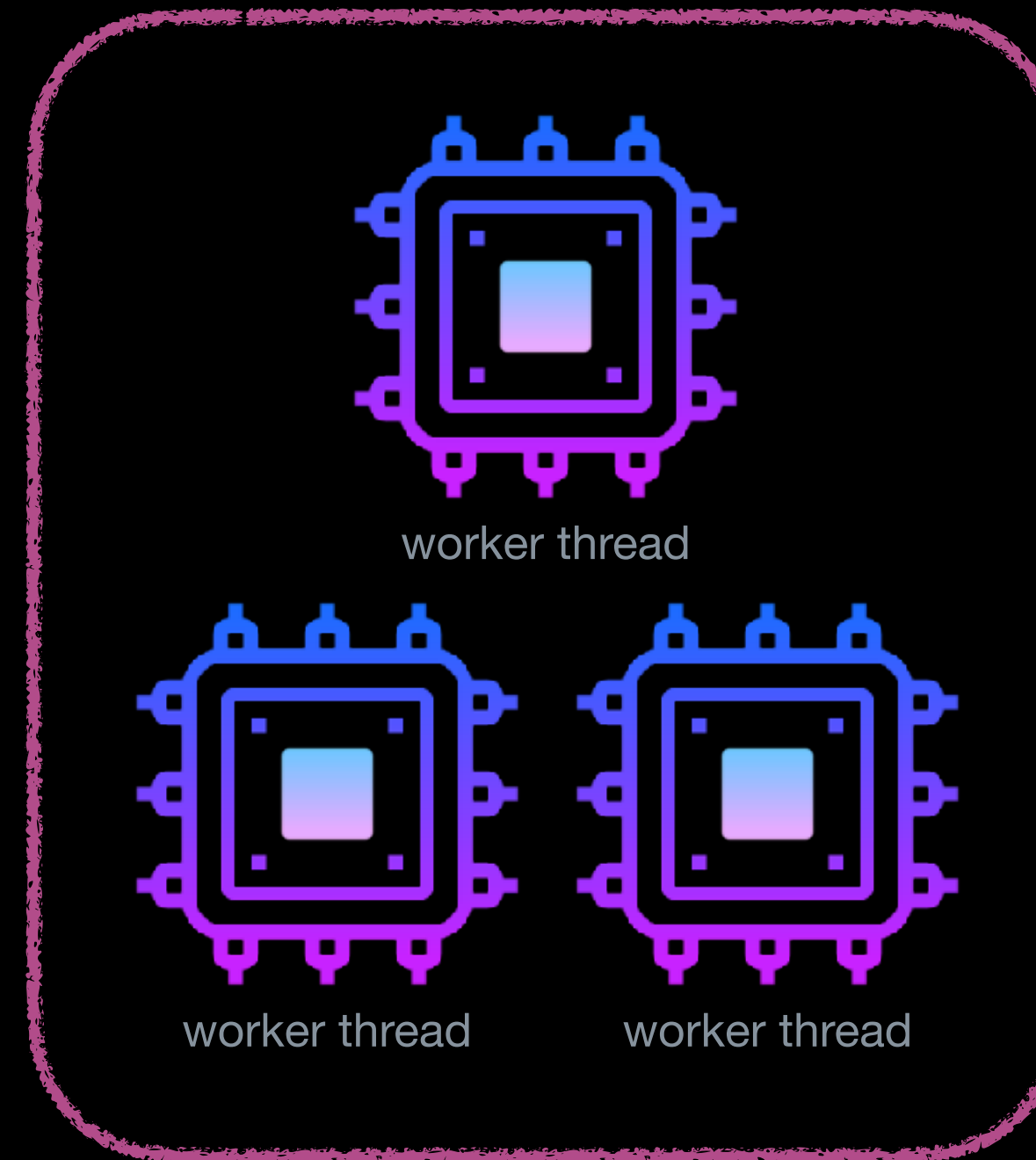
Completed Tasks

dispatch queue



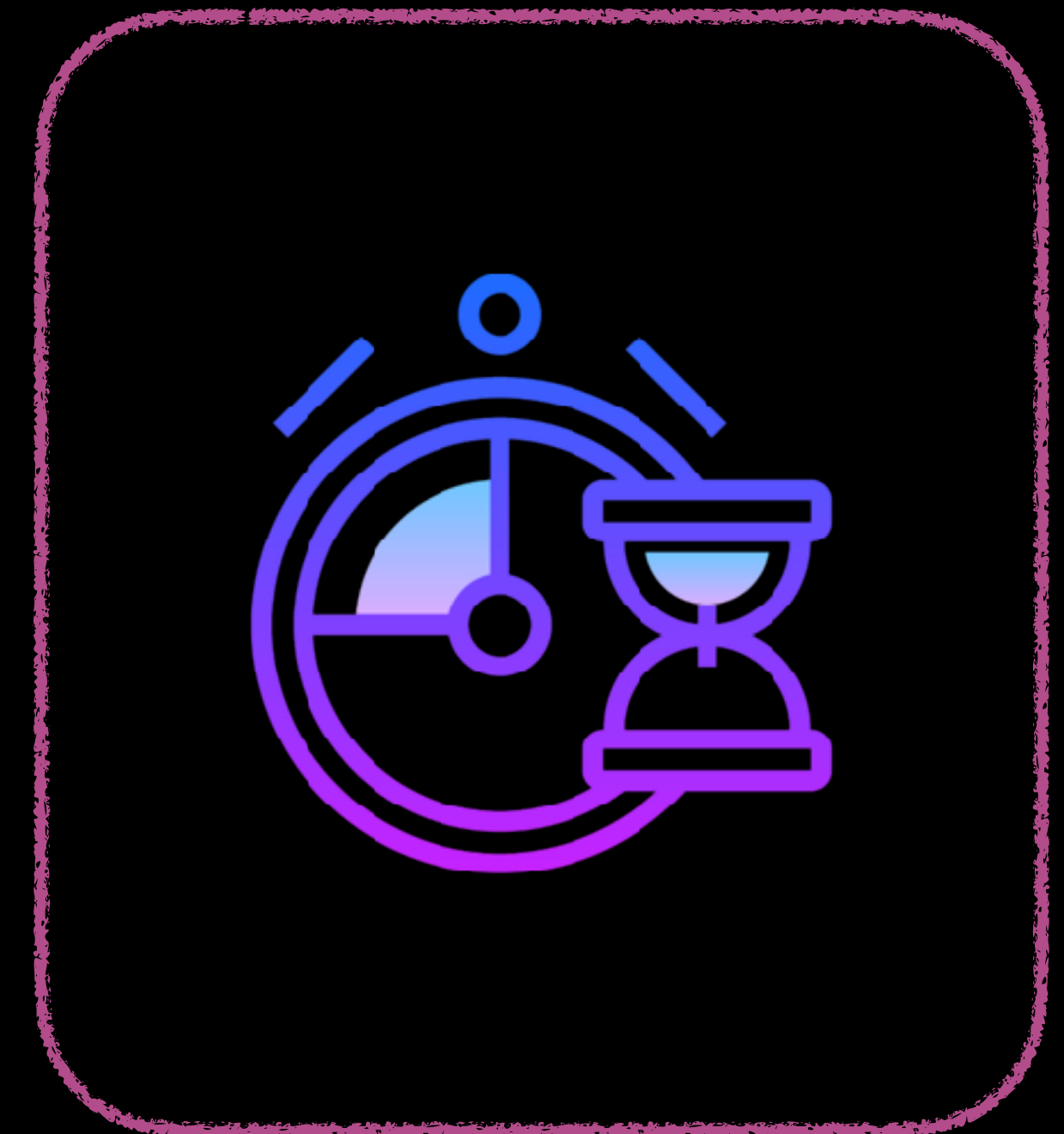
many tasks queued

thread pool

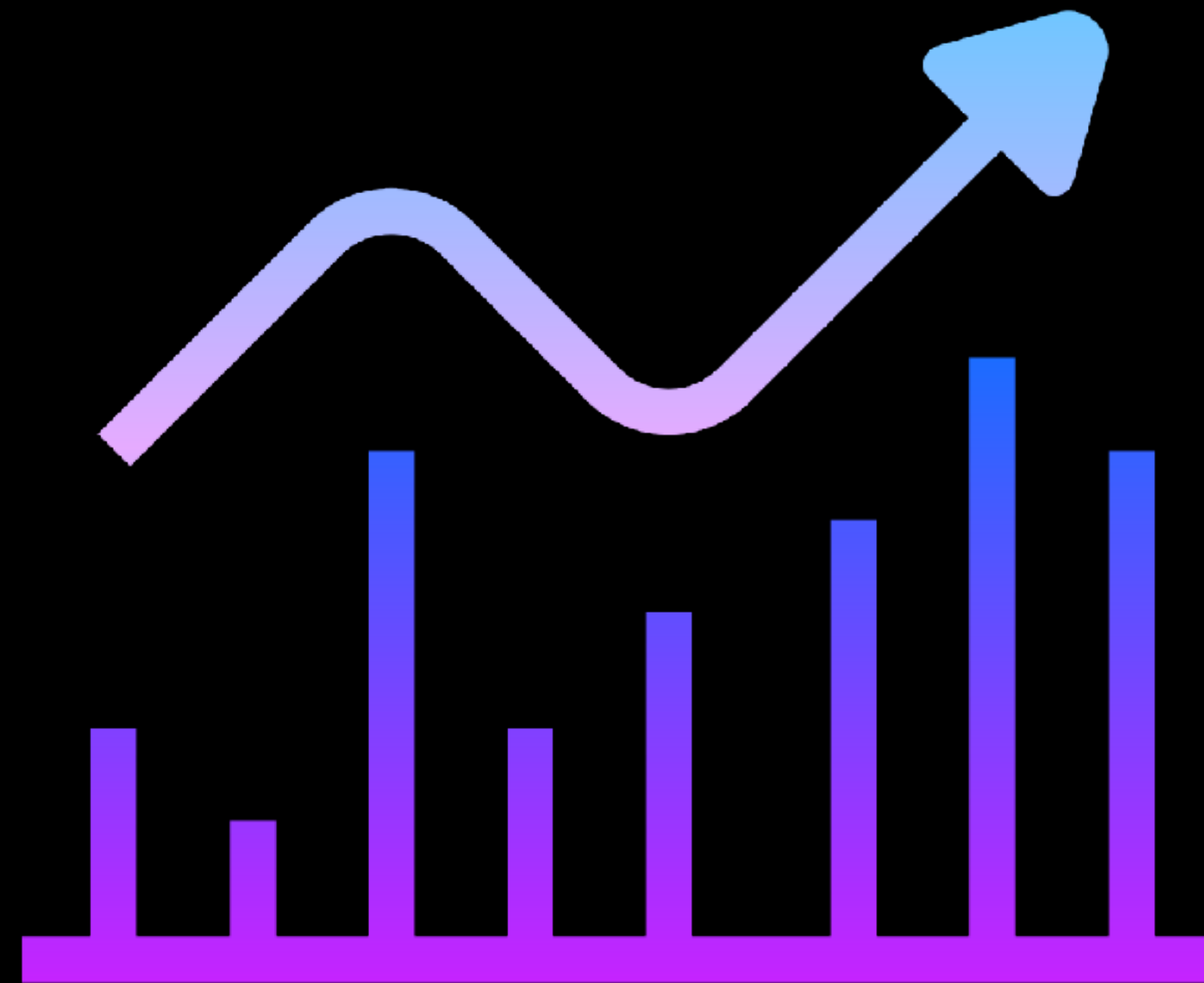


all threads busy (blocking)

starving task / component

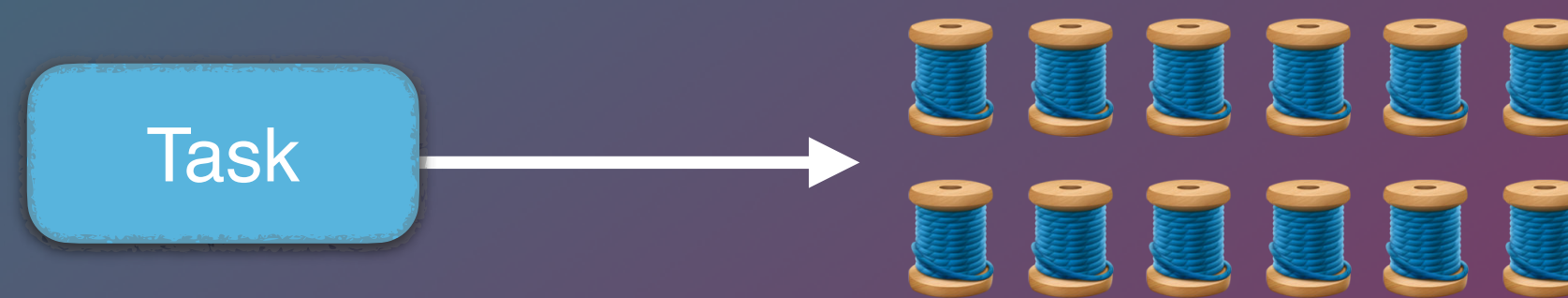


starved of resources



historical context

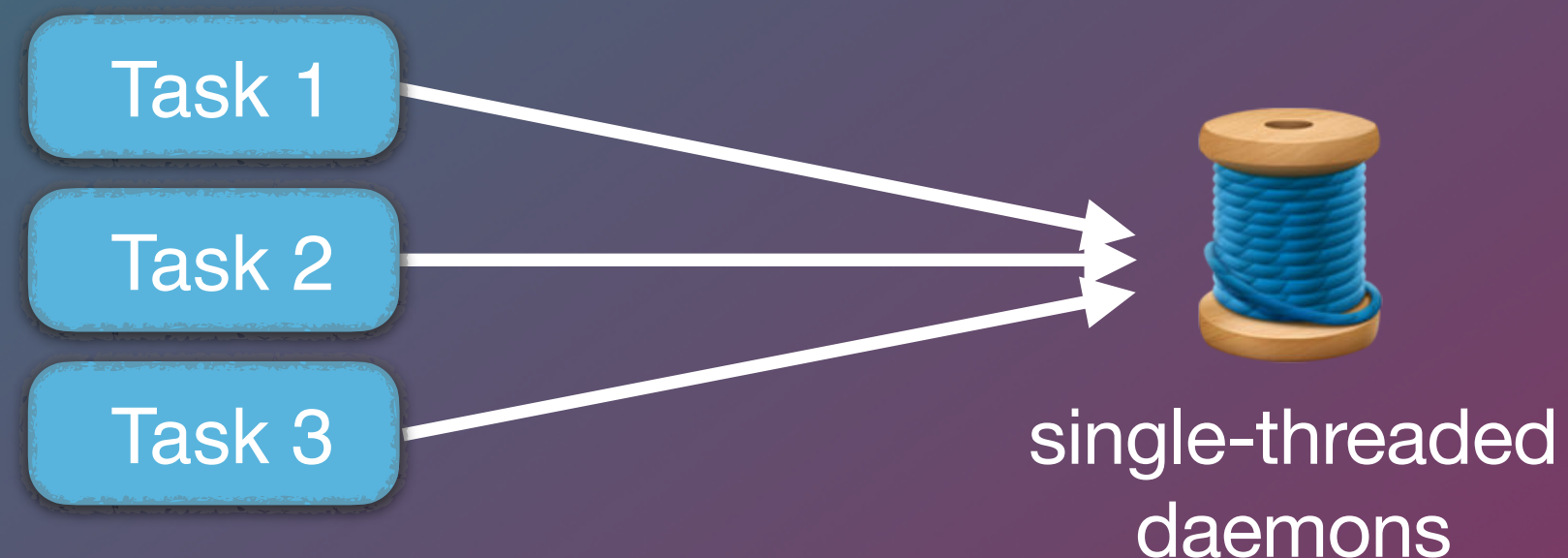
abandoned API (macOS 10.7)



security transforms:
new queue/thread per task

caused severe thread proliferation

rewrite (iOS 12)



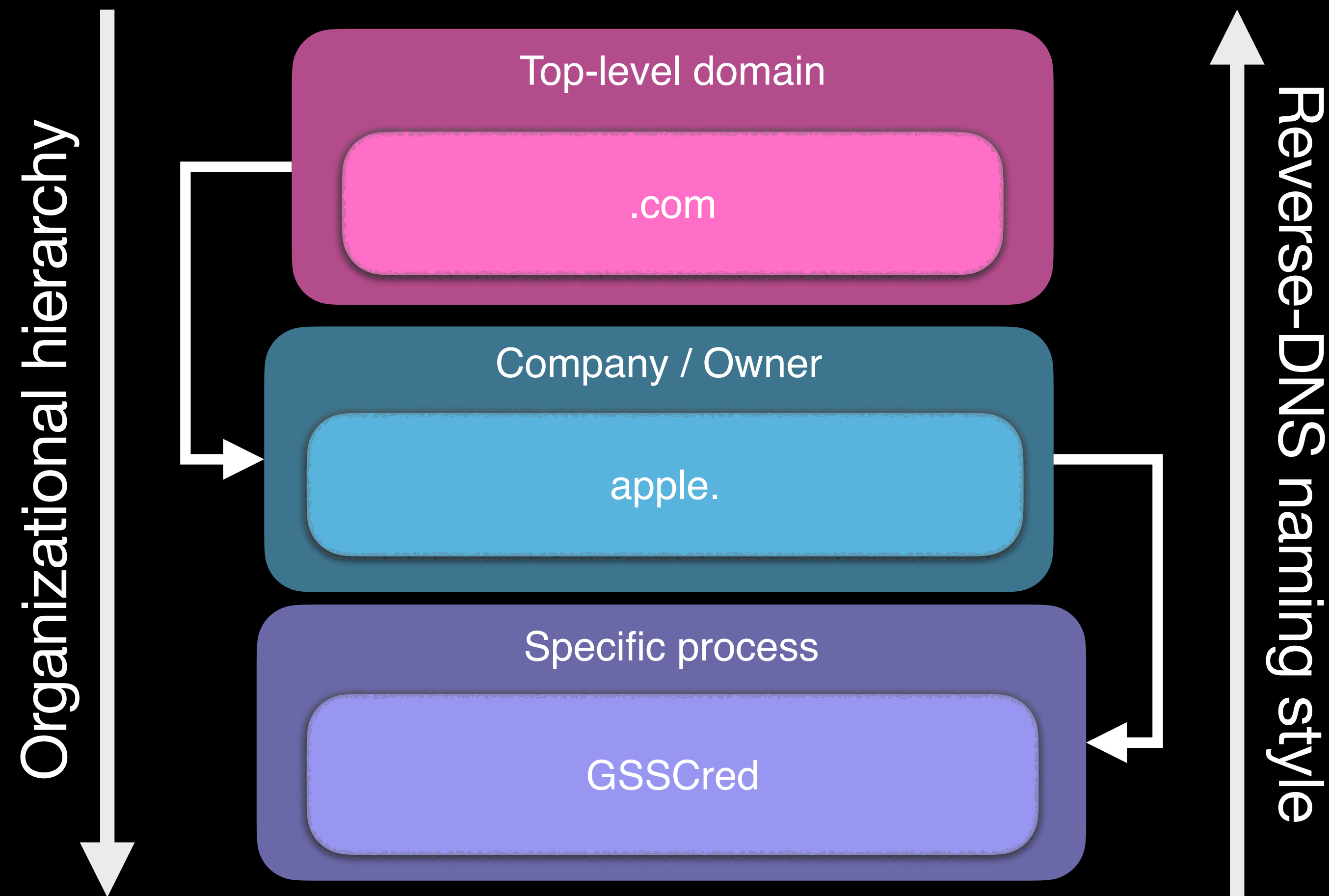
improved performance, unconstrained
concurrency can backfire.

CVE-2018-4331

\$ com.apple.GSSCred

reverse-dns identifiers

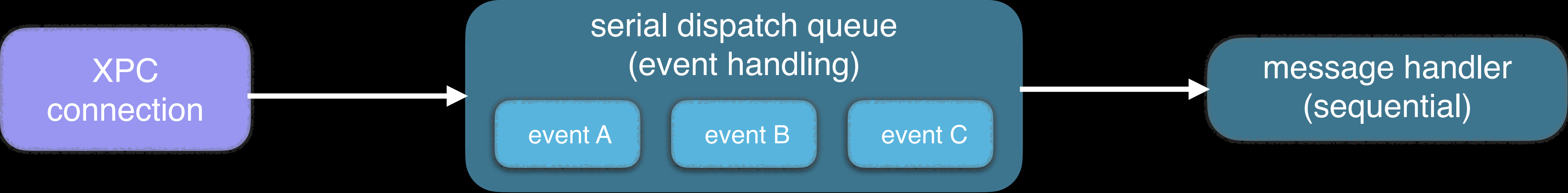
com.apple.GSSCred



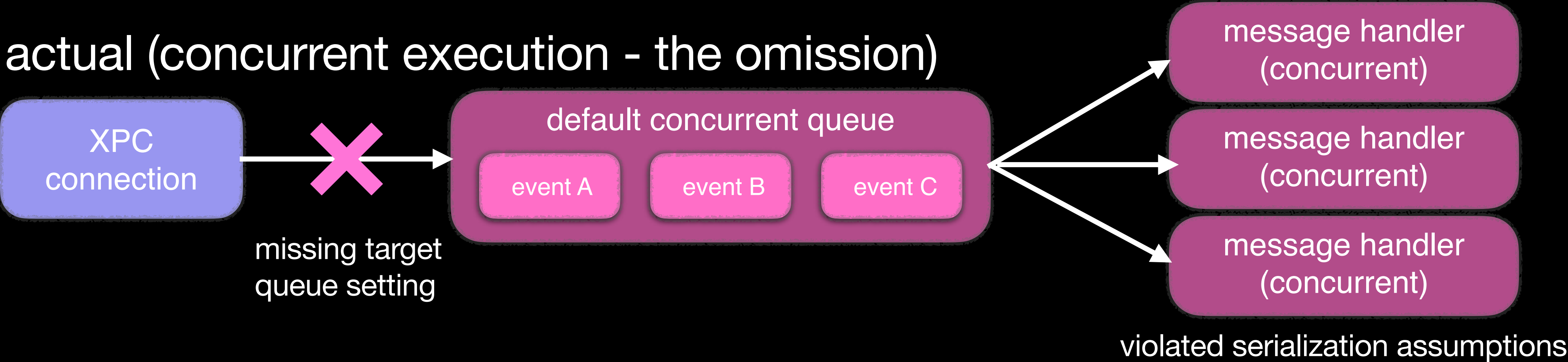
serialization failure

XPC connection omission

intended (serial execution)



actual (concurrent execution - the omission)



That brings us to *today*

auditing

privileged XPC services

- **Concurrency risks:** unmanaged XPC paths, omitted queues, and cross-queue blocking
- **State vulnerabilities:** unsynchronized mutable state across handlers
- **Behavioral anomalies:** XPC messaging bursts
- **Yikes indicators:** crashes, daemon restarts, or anomalous “x” (e.g.,) GSSCred activity

\$ sandboxing

AppleDeveloper

Get StartedPlatformsTechnologiesCommunityDocumentationDownloadsSupport

Documentation

Language: Objective-C

< All Technologies

XPC

Essentials

XPC updates

Interprocess communication

Creating XPC services

xpc_listener_t

xpc_session_t

Tasks

XPC activities

Events

XPC events

Additional types

XPC objects

launchd

Utilities

XPC connections

Reference

Macros

Protocols

OS_xpc_peer_requirement

Framework

XPC

Access a low-level interprocess communication mechanism.

iOS 17.4+ | iPadOS 17.4+ | Mac Catalyst 13.0+ | macOS 10.10+

Overview

XPC provides a lightweight mechanism for basic interprocess communication. It allows you to create lightweight helper tools, called *XPC services*, that perform work on behalf of your app. The `launchd` system daemon manages these services, launching them on demand, shutting them down when idle, and restarting them if they crash. Benefits of XPC services include:

- Centralize work from multiple processes or mediate access to a shared resource.
- Delegate work so it continues beyond a client’s life cycle.
- Privilege isolation to narrow the scope of access for different functionality.

Clients that make use of these services rely on peer-to-peer XPC connections to communicate across process boundaries. There are two sides to each connection. One side, the *listener* or server, responds to incoming connection requests and performs tasks. The other side, the client, initiates connections to an XPC service by creating a *session* with a listener. Once a client establishes a connection to the listener, it sends messages and receives replies from the service.

The type of XPC service you build depends on the requirements of the work it performs. The following table summarizes the types of services available and some differences in how they behave:

Service	Process Environment
---------	---------------------

\$ brew uninstall threats

detection patterns

static code review signals

detection patterns

static code review signals

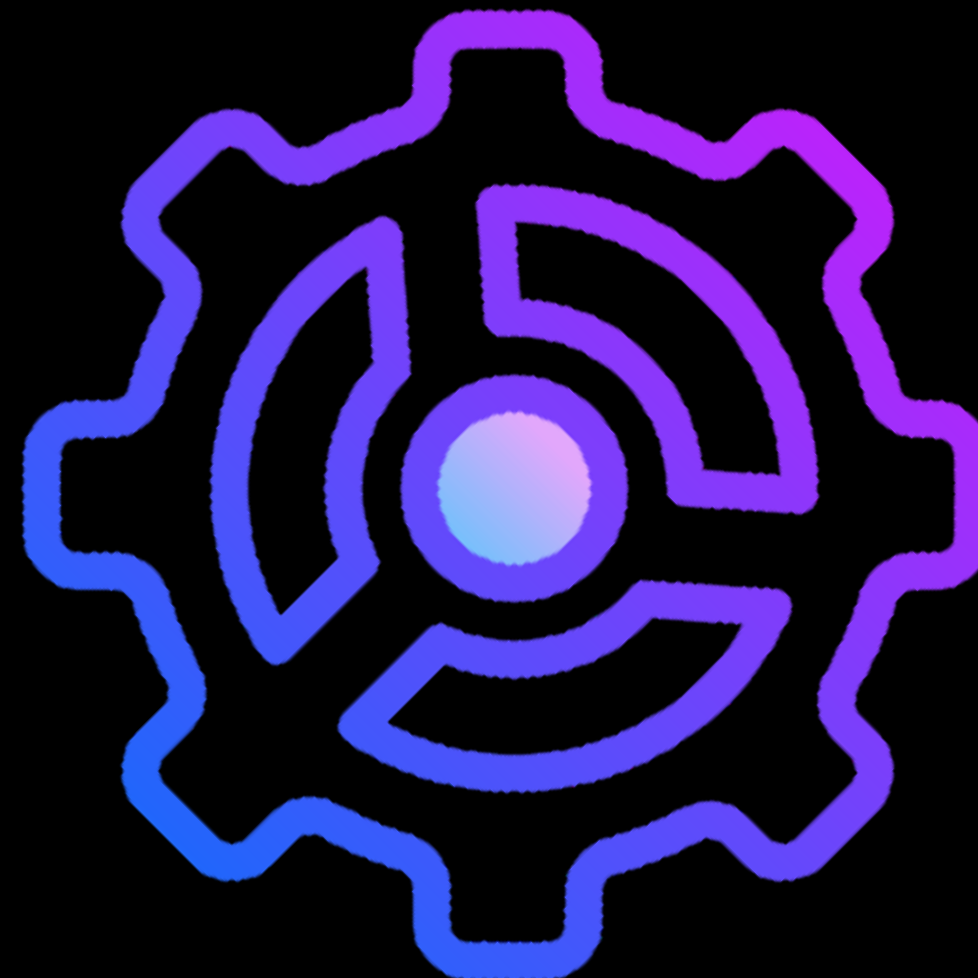
- **XPC connection queueing**: Ensure `xpc_connection_set_target_queue()` is always called for accepted connections
- **Serial vs concurrent patterns**: Detect improper assumptions about serial execution in concurrent environments
- **Synchronous calls**: Flag `dispatch_sync` use and require justification in code review
- **Shared mutable state without synchronization**: Identify globals accessed without locks or serial queue funnels

\$./telemetry-signals

Crash patterns

Thread churn spikes

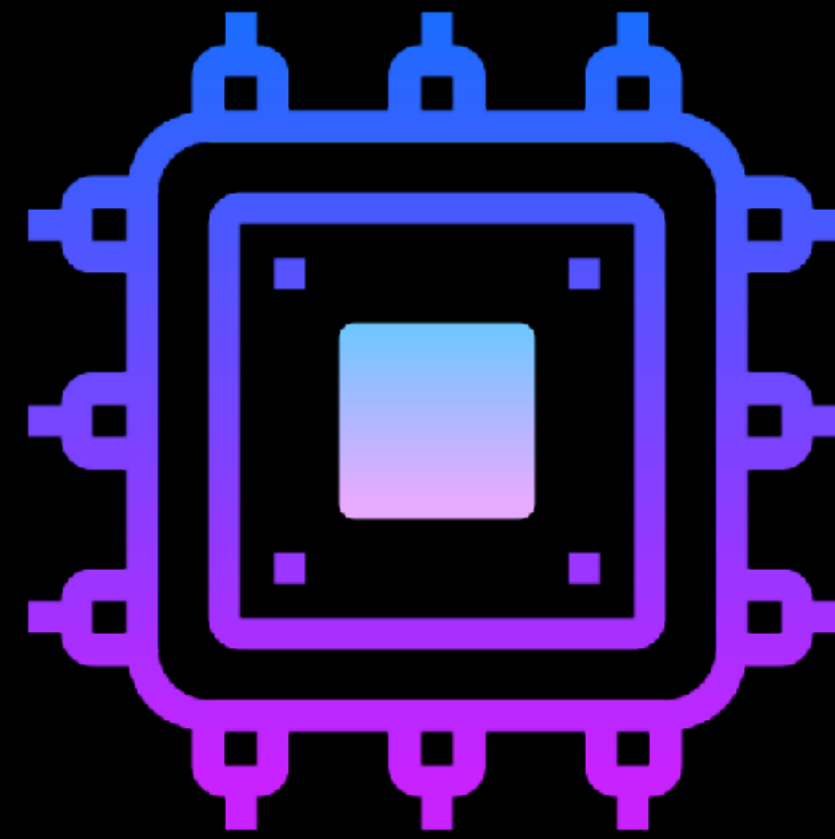
Queue backlogs



Crash patterns

Thread churn spikes

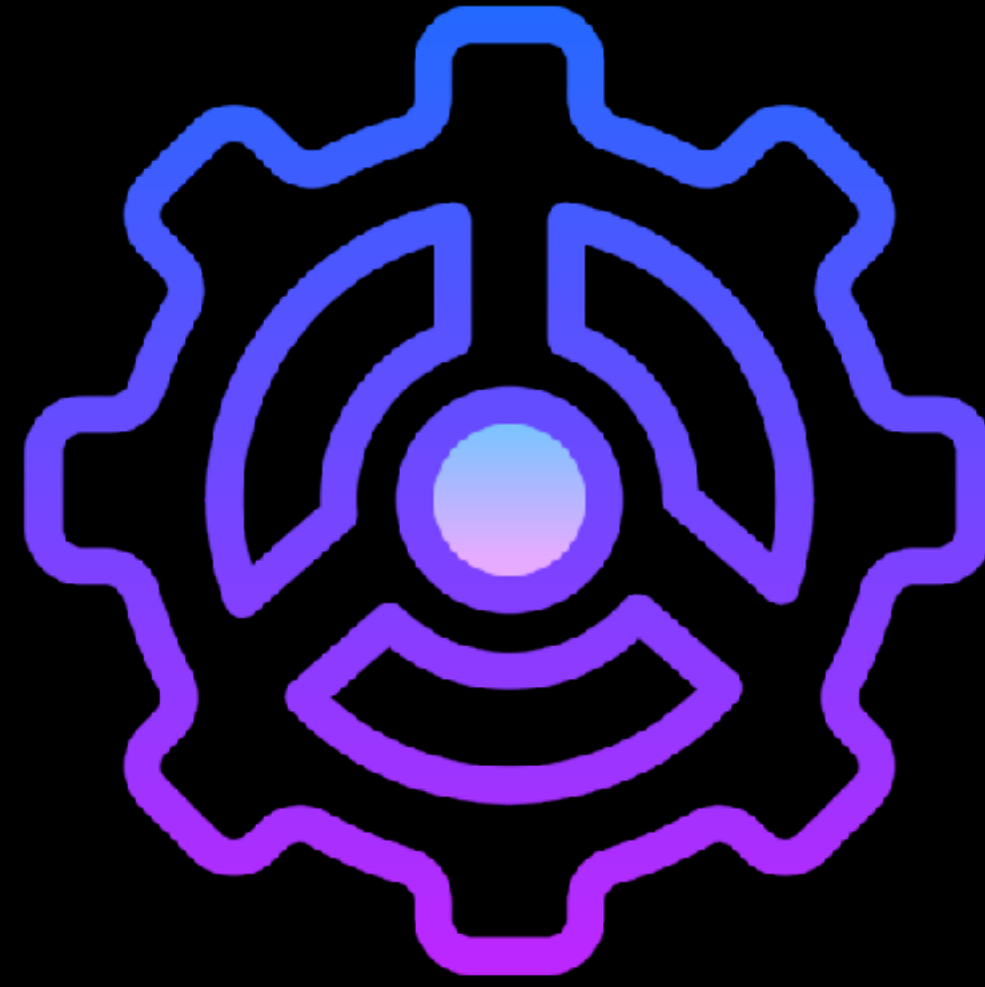
Queue backlogs



Crash patterns

Thread churn spikes

Queue backlogs



\$ cd /behavior

Elevated Synchronization

Expected Parallelism

Behavior

Rate Limiting



Baseline

Drain Time

Static Review

Telemetry

Behavioral Detections

Static Review

Telemetry

Behavioral Detections

Static Review

Telemetry

Behavioral Detections


```
$ ./conclusion.sh
```

Conclusion

- How GCD works and common concurrency hazards
- Priority inversion and deadlock pitfalls with synchronous queue usage
- The exploit against `com.apple.GSSCred` showing how a queue misconfiguration can lead to code execution in privileged services
- Detection strategies: code review patterns, telemetry signals, and architectural patterns

\$ touch future.txt

👤 Subscribe to [ret]2read: An OS Internals Newsletter!

Interested in macOS internals, reverse engineering, or open-source tools for exploring obscure systems?

Starting in September 2025, I'll be publishing a *monthly* newsletter on Apple security research, sharing what I'm reading, and how I'm applying it to my work.

If that sounds cool, consider subscribing. 🍏🔥🌟

✓ Subscribed



<https://oliviagallucci.com/newsletter/>

\$ echo
"thanks"

👤 Subscribe to [ret]2read:
An OS Internals
Newsletter!

Interested in macOS internals, reverse engineering, or
open-source tools for exploring obscure systems?

Starting in September 2025, I'll be publishing a *monthly*
newsletter on Apple security research, sharing what I'm
reading, and how I'm applying it to my work.

If that sounds cool, consider subscribing. 🍷🎉🥂

✓ Subscribed



<https://oliviagallucci.com/newsletter/>

\$ echo
"questions?"

👤 Subscribe to [ret]2read:
An OS Internals
Newsletter!

Interested in macOS internals, reverse engineering, or
open-source tools for exploring obscure systems?

Starting in September 2025, I'll be publishing a *monthly*
newsletter on Apple security research, sharing what I'm
reading, and how I'm applying it to my work.

If that sounds cool, consider subscribing. 🍏🍰🌟

✓ Subscribed



<https://oliviagallucci.com/newsletter/>

\$ shutdown

👤 Subscribe to [ret]2read: An OS Internals Newsletter!

Interested in macOS internals, reverse engineering, or open-source tools for exploring obscure systems?

Starting in September 2025, I'll be publishing a *monthly* newsletter on Apple security research, sharing what I'm reading, and how I'm applying it to my work.

If that sounds cool, consider subscribing. 🍷🎉🥂

✓ Subscribed



<https://oliviagallucci.com/newsletter/>